LingoPair

Database Design

Domain-Driven Design Analysis + PostgreSQL DDL

Vernon's Four Rules Applied · 6 Bounded Contexts · 32 Tables · 10 Enums · 53 Indexes

identity	sessions	messaging	social	marketplace	reporting
6 tables	10 tables	4 tables	3 tables	3 tables	2 tables

Part 1 — Domain-Driven Design Research

Domain-Driven Design (DDD) is an approach to software development that aligns the codebase with the business domain. It distinguishes strategic design (how to divide the system into cohesive subsystems) from tactical design (how to model behaviour inside each subsystem). This section grounds every design decision in Vaughn Vernon's authoritative framework from Implementing Domain-Driven Design.

1.1 The Four Aggregate Rules (Vernon)

Vernon identifies four rules of thumb that, when applied correctly, produce aggregates with clear transactional boundaries, minimal lock contention, and room for independent scalability. Every table in Part 2 is justified against these rules.

Rule	Summary	What it means in practice	Applied in LingoPair
Rule 1	Model true invariants in consistency boundaries	Only bundle objects together whose constraints MUST be enforced atomically in one DB transaction. Separate everything else.	Each aggregate has a version column + a PostgreSQL trigger that rejects stale writes. Key business rules (paid ↔ price, status lifecycle, no self-follow) are DB-level CHECK constraints.
Rule 2	Design small aggregates	Start with each entity as its own aggregate root. Only pull in children whose invariants are inseparable from the root — not for convenience.	Participant is separate from Room (mute toggles at high frequency — different transaction rhythm). DirectMessage is separate from Conversation (high-volume inserts). HandRaiseRequest has its own lifecycle.
Rule 3	Reference other aggregates by identity only	Across aggregate (and BC) boundaries, store only the UUID. No FK, no JOIN in command handlers. Cross-aggregate data assembled in read-model views only.	Every cross-BC reference is a UUID with a comment: "identity.users ID only (Rule 3)". No REFERENCES clause crosses a schema boundary. Joins only in identity.people_directory view.
Rule 4	Use eventual consistency outside the boundary	Counts, stats, and projections that span aggregates are updated asynchronously via domain events, not in the same transaction.	reporting.user_stats and reporting.session_logs are pure read models updated by event handlers. Each write BC has an outbox_events table for at-least-once delivery of domain events.

1.2 Bounded Contexts

A Bounded Context is an explicit boundary within which a particular domain model applies. The same term (e.g. "User") may mean different things in different contexts. Fowler's rule: draw boundaries around teams, their vocabulary, and their model. We map each BC to its own PostgreSQL schema — the strongest isolation mechanism available in a single database.

Schema	Subdomain type	Responsibility	Aggregate roots
identity	Core domain	User accounts, handles, language profiles, privacy controls	User (+ language, topic, privacy, availability children)

Schema	Subdomain type	Responsibility	Aggregate roots
sessions	Core domain	Room lifecycle: browse → create → live session → ended. Chat, hand raises, participants.	Room · Participant · ChatMessage · HandRaiseRequest · WaitlistEntry · RoomReminder
messaging	Supporting	Direct 1-on-1 messaging, message requests, read receipts	Conversation · DirectMessage
social	Generic	Follow graph, notification bell (US-005), activity feed (US-040/041)	Follow · Notification · ActivityEntry
marketplace	Supporting	Teacher credentials, 1-on-1 bookings, reviews (US-042/045)	TeacherProfile · Review · Booking
reporting	Generic	Read-model projections updated by domain events (Rule 4). Never written by commands.	UserStats (projection) · SessionLog (audit)

1.3 Context Map

A context map documents how bounded contexts interact. LingoPair uses three relationship patterns:

Customer–Supplier	sessions BC publishes domain events (RoomEnded, ParticipantLeft). reporting BC consumes them to update UserStats and SessionLogs. sessions = upstream supplier; reporting = downstream customer.
Conformist	messaging and social BCs reference identity.users by UUID only. They conform to the identity model without owning User data.
Shared Kernel	All BCs share 10 PostgreSQL enum types (cefr_level, room_status, etc.) — a narrow shared kernel of ubiquitous vocabulary.
Anti-Corruption Layer	identity.people_directory is a read-only cross-BC view. It shields the UI layer from internal schema changes in reporting and marketplace, acting as an ACL for the query side.

1.4 Ubiquitous Language Glossary

Terms used consistently across schema column names, user stories, and code. Wherever the name appears in the DDL it carries the meaning defined here.

Room	A time-bounded live audio session. Lifecycle: scheduled → live → ended. Aggregate root in sessions BC. Status transitions are enforced by a DB CHECK constraint.
Participant	A User inside a live Room. Has exactly one role at a time: listener | speaker | host. Role transitions are domain events. Separate aggregate from Room (Rule 2).
Orb	UI-only concept — visual avatar card for a speaker on stage. Not persisted; derived from Participant.role and Participant.is_muted at query time.
Hand Raise	A Listener's request to become a Speaker. Modelled as HandRaiseRequest (own aggregate, own lifecycle: raised → approved | denied | cancelled).
Correction	A chat or DM message flagged as a language correction (is_correction = TRUE). Displayed with a distinct fire-bordered bubble.
Follow	A directed social graph edge (follower → followee). Not symmetric. Two rows needed for mutual following. Invariant: no self-follow.

Message Request	A Conversation where request_status = 'pending'. The input bar is locked until accepted. Domain event: RequestAccepted.
Session	One contiguous period a User spends in a Room, persisted as a reporting.session_logs row. Source of truth for hours_spoken and day_streak computation.
Stats Bar	Profile read-model: 4-cell card (Sessions / Hours / Day streak / Avg rating). Projection from reporting.user_stats; updated eventually (Rule 4).
Teacher Profile	Extended profile for user_role = 'teacher'. Aggregate root in marketplace BC. avg_rating and review_count are eventual projections from Review events.

1.5 Tactical Design — Aggregate Boundaries

The key tactical decision in DDD is where to draw aggregate boundaries. We follow Vernon's "start small" advice: every entity begins as its own aggregate root, then children are pulled in only when there is a true transactional invariant that cannot be deferred. The following cards document every aggregate root and the reasoning behind its boundary.

AGGREGATE ROOT **User**

Table: `identity.users`

Children: user_languages, user_topics, privacy_settings, user_availability

Value Objects: Handle (CITEXT unique) · CEFRLevel (enum) · LanguageProgress (progress_pct + months_studied)

True invariants (Rule 1):

- bio ≤ 220 chars — CHECK constraint (US-036).
- display_name 1–80 chars — CHECK constraint.
- handle is globally unique and case-insensitive — UNIQUE on CITEXT column.
- Exactly 5 privacy toggles, stored as one row, created atomically with the User (US-043).
- Optimistic lock: version INTEGER + enforce_version_bump trigger.

Domain events: UserRegistered, ProfileUpdated, LanguageAdded, LanguageRemoved, PrivacySettingChanged

AGGREGATE ROOT **Room**

Table: `sessions.rooms`

Children: room_languages (max 3), room_levels, room_topics

Value Objects: RoomStatus (enum — one-way lifecycle) · RoomAccess (free | paid) · DurationPreset (enum)

True invariants (Rule 1):

- title 1–80 chars, description ≤ 280 chars — CHECK constraints (US-015).
- access = paid ⟺ price_per_person IS NOT NULL — ck_rooms_paid_price (US-018).
- Status/date consistency: ck_rooms_status_dates ensures started_at/ended_at match status.
- duration_seconds is GENERATED ALWAYS AS — computed from started_at/ended_at, immutable.
- max_participants ≥ 2 — CHECK constraint. Application enforces language count ≤ 3 (US-016).

Domain events: RoomCreated, RoomStarted, RoomEnded

AGGREGATE ROOT **Participant**

Table: `sessions.participants`

Children: (none — smallest valid aggregate)

Value Objects: ParticipantRole (listener | speaker | host)

True invariants (Rule 1):

- Separate from Room (Rule 2): is_muted and role change at high frequency in live sessions. A single Room row would be locked on every mute toggle for every participant simultaneously.
- (room_id, user_id) UNIQUE — one row per user per room.
- hand_raised_at tracks current hand-raise state inline; HandRaiseRequest tracks the resolution lifecycle.
- Optimistic lock: version + trigger guard against concurrent mute races.

Domain events: ParticipantJoined, ParticipantLeft, RoleChanged, ParticipantMuted, ParticipantUnmuted, ParticipantKicked

AGGREGATE ROOT **ChatMessage**

Table: `sessions.chat_messages`

Children: chat_reactions

Value Objects: (immutable after creation)

True invariants (Rule 1):

- ck_chat_has_content: body IS NOT NULL OR voice_url IS NOT NULL.
- ck_chat_correction_text: NOT (is_correction AND voice_url IS NOT NULL) — corrections are text only.
- Immutable after insert — corrections are new messages, not edits. No UPDATE on this table.

Domain events: ChatMessageSent, ReactionToggled

AGGREGATE ROOT **HandRaiseRequest**

Table: `sessions.hand_raise_requests`

Children: (none)

Value Objects: Resolution (approved | denied | cancelled)

True invariants (Rule 1):

- Separate from Participant (Rule 2): resolution happens in a different transaction from the role promotion that follows approval.
- ck_hrr_resolved: resolved_at and resolution must be both NULL or both set.

Domain events: HandRaised, HandApproved, HandDenied, HandCancelled

AGGREGATE ROOT **Conversation**

Table: `messaging.conversations`

Children: conversation_members (per-user read state)

Value Objects: RequestStatus (pending | accepted | declined)

True invariants (Rule 1):

- participant_a ≠ participant_b — ck_conv_different_users.
- The pair (a, b) is unique using UNIQUE(LEAST(a::TEXT, b::TEXT), GREATEST(…)) — no duplicate threads regardless of insertion order.
- requested_by must equal participant_a or participant_b — ck_conv_requester_is_member.
- ConversationMember is a child (same tx): marking a message read must atomically decrement unread_count and set last_read_at.

Domain events: ConversationCreated, RequestAccepted, RequestDeclined

AGGREGATE ROOT **DirectMessage**

Table: `messaging.direct_messages`

Children: message_reactions

Value Objects: MessageStatus (sent | delivered | read)

True invariants (Rule 1):

- Separate from Conversation (Rule 2): messages arrive at high volume and are immutable. Keeping them inside Conversation would lock the conversation row on every send.
- ck_dm_has_content: body, voice_url, or attachment_url must be set.
- ck_dm_read_consistency: status = 'read' ↔ read_at IS NOT NULL.

Domain events: MessageSent, MessageRead, ReactionToggled

AGGREGATE ROOT **TeacherProfile**

Table: `marketplace.teacher_profiles`

Children: (none — avg_rating/review_count are eventual projections)

Value Objects: HourlyRate (NUMERIC 8,2) · StarRating projection (eventual)

True invariants (Rule 1):

- hourly_rate ≥ 0 — CHECK constraint.
- avg_rating and review_count are NOT true invariants — they are eventual projections updated by ReviewSubmitted event (Rule 4). They are denormalised for display performance only.
- Exists only for users with user_role = 'teacher' — enforced at application layer.

Domain events: TeacherProfileCreated, TeacherProfileUpdated

AGGREGATE ROOT **Review**

Table: `marketplace.reviews`

Children: (none)

Value Objects: StarRating (SMALLINT 1–5)

True invariants (Rule 1):

- rating BETWEEN 1 AND 5 — CHECK constraint.
- UNIQUE (subject_id, author_id) — one review per author per subject.
- subject_id ≠ author_id — ck_review_no_self.

Domain events: ReviewSubmitted

AGGREGATE ROOT **Booking**

Table: `marketplace.bookings`

Children: (none)

Value Objects: BookingStatus (pending | confirmed | cancelled | completed)

True invariants (Rule 1):

- teacher_id ≠ student_id — ck_booking_no_self.
- amount ≥ 0 — CHECK constraint.
- status must be one of four defined values — ck_booking_status.
- room_id is a loose reference to sessions.rooms by ID only — no FK cross-BC (Rule 3).

Domain events: BookingRequested, BookingConfirmed, BookingCancelled, BookingCompleted

1.6 Value Objects vs Entities — Decision Table

Value objects are immutable, equality-by-value, and have no independent lifecycle. Entities have identity and mutable state. This distinction drives whether something gets its own table row with a PK, or is embedded directly.

Handle	Value object embedded in identity.users. Enforced unique by DB. Immutable after creation.
CEFRLevel	Value object enum. Changing level = replacing the value, not updating an entity.
LanguageProgress	Value object (progress_pct + months_studied) embedded in user_languages row.
RoomStatus	Value object enum. Status transitions are domain events, not entity mutations.
DurationPreset	Value object enum for scheduling presets. No identity of its own.
HandRaiseRequest	Entity — has a UUID PK, a lifecycle, and a resolution. Mutable.
ChatMessage	Entity — has UUID, immutable body, sender, timestamp. Reactions are child collection.
Follow	Lightweight entity — identity is composite PK (follower_id, followee_id). Exists or does not.
MessageStatus	Value object enum embedded in direct_messages. Updated in-place on delivery/read.
StarRating	Value object (SMALLINT 1–5) inside a Review entity. Equality by value.

Part 2 — PostgreSQL DDL

The DDL implements the domain model from Part 1. Each PostgreSQL schema corresponds to one bounded context. Within a schema, tables are ordered by dependency. All DDL executes within a single transaction (BEGIN … COMMIT).

Target RDBMS	PostgreSQL 15+
Extensions	pgcrypto (gen_random_uuid), citext (case-insensitive handles/email), pg_trgm (trigram full-text search)
PK strategy	UUID v4 via gen_random_uuid() — no sequential ID leakage, safe for distributed inserts
Naming	snake_case throughout; schema-qualified tables (schema.table_name)
Optimistic concurrency	version INTEGER on every aggregate root + enforce_version_bump() trigger (Rule 1)
Cross-BC references	UUID only, no REFERENCES clause crossing schemas (Rule 3). Comment on every such column.
Eventual consistency	outbox_events table per write BC; partial index WHERE NOT published (Rule 4)
Read models	reporting schema only — no domain commands touch it. Projections rebuilt from events.
Computed columns	GENERATED ALWAYS AS STORED for duration_seconds (immutable derived fact)

Extensions, Schemas & Shared Kernel Enums

```
-- Extensions
-- ------------------------------------------------------------
CREATE EXTENSION IF NOT EXISTS pgcrypto;  -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS citext;    -- case-insensitive text
CREATE EXTENSION IF NOT EXISTS pg_trgm;   -- trigram text search

-- ------------------------------------------------------------
-- Schemas — one per bounded context (Rule 3: hard isolation)
-- ------------------------------------------------------------
CREATE SCHEMA IF NOT EXISTS identity;
CREATE SCHEMA IF NOT EXISTS sessions;
CREATE SCHEMA IF NOT EXISTS messaging;
CREATE SCHEMA IF NOT EXISTS social;
CREATE SCHEMA IF NOT EXISTS marketplace;
CREATE SCHEMA IF NOT EXISTS reporting;

-- ============================================================
-- SHARED KERNEL — Value Object types
-- Narrow shared vocabulary; all BCs reference these enums.
-- ============================================================
CREATE TYPE cefr_level        AS ENUM ('A1','A2','B1','B2','C1','C2','native','all_levels');
CREATE TYPE user_role         AS ENUM ('learner','exchange','teacher');
CREATE TYPE room_status       AS ENUM ('scheduled','live','ended');
CREATE TYPE room_access       AS ENUM ('free','paid');
CREATE TYPE participant_role  AS ENUM ('listener','speaker','host');
CREATE TYPE duration_preset   AS ENUM ('30min','45min','1hour','90min','2hours');
CREATE TYPE topic_tag         AS ENUM (
    'tech_it','business','travel','drama_film','music_pop',
    'news_culture','exam_prep','debate','kids_families',
    'food_cooking','sports_fitness','gaming'
```

```
);
CREATE TYPE availability_slot AS ENUM ('weekdays','weekends','mornings','evenings');
CREATE TYPE msg_status        AS ENUM ('sent','delivered','read');
CREATE TYPE request_status    AS ENUM ('pending','accepted','declined');
```

BC: identity — User aggregate

```
-- BC: identity
--
-- AGGREGATE ROOT: User
-- True invariants (Rule 1):
--    · bio ≤ 220 chars (US-036)
--    · display_name 1-80 chars
--    · handle is globally unique and case-insensitive
--    · Exactly 5 privacy toggles, always set as a unit (US-043)
--
-- Child entities (same tx as root — Rule 2):
--    · UserLanguage   — languages the user speaks/learns (US-037)
--    · UserTopic      — interest chips on profile (US-038)
--    · PrivacySetting — five on/off toggles (US-043)
--    · UserAvailability — time-slot preferences (US-047)
--
-- NOT children (separate aggregates, Rule 2):
--    · Follow, Notification → social BC (different lifecycle)
--    · Stats → reporting BC (eventual, Rule 4)
-- ============================================================

CREATE TABLE identity.users (
    id              UUID        NOT NULL DEFAULT gen_random_uuid(),
    handle          CITEXT      NOT NULL,
    email           CITEXT      NOT NULL,
    password_hash   TEXT        NOT NULL,

    -- Embedded Value Objects
    display_name    TEXT        NOT NULL
                                CONSTRAINT ck_user_name_len
                                CHECK (char_length(display_name) BETWEEN 1 AND 80),
    bio             TEXT        CONSTRAINT ck_user_bio_len
                                CHECK (bio IS NULL OR char_length(bio) <= 220),
    avatar_url      TEXT,
    cover_url       TEXT,
    city            TEXT,
    user_role       user_role   NOT NULL DEFAULT 'learner',

    -- Presence (set by heartbeat, not a domain command)
    is_online       BOOLEAN     NOT NULL DEFAULT FALSE,
    last_seen_at    TIMESTAMPTZ,

    -- Optimistic lock — guards the consistency boundary (Rule 1)
    version         INTEGER     NOT NULL DEFAULT 1,

    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_users         PRIMARY KEY (id),
    CONSTRAINT uq_users_handle  UNIQUE (handle),
    CONSTRAINT uq_users_email   UNIQUE (email)
);

-- Child: UserLanguage (owned by User aggregate)
-- Invariant: one row per (user, language); display_order managed by root.
CREATE TABLE identity.user_languages (
    id              UUID        NOT NULL DEFAULT gen_random_uuid(),
    user_id         UUID        NOT NULL,   -- FK inside BC only (Rule 3 within BC)
```

```
    language_code   CHAR(3)     NOT NULL,
    language_name   TEXT        NOT NULL,
    flag_emoji      TEXT        NOT NULL,
    level           cefr_level  NOT NULL,
    is_native       BOOLEAN     NOT NULL DEFAULT FALSE,
    progress_pct    SMALLINT    NOT NULL DEFAULT 0
                                CONSTRAINT ck_lang_progress CHECK (progress_pct BETWEEN 0 AND
100),
    months_studied  SMALLINT    NOT NULL DEFAULT 0
                                CONSTRAINT ck_lang_months CHECK (months_studied >= 0),
    display_order   SMALLINT    NOT NULL DEFAULT 0,
    created_at      TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_user_languages         PRIMARY KEY (id),
    CONSTRAINT uq_user_language_per_user UNIQUE (user_id, language_code),
    CONSTRAINT fk_user_languages_user    FOREIGN KEY (user_id)
        REFERENCES identity.users (id) ON DELETE CASCADE
);

-- Child: UserTopic (US-038)
CREATE TABLE identity.user_topics (
    user_id     UUID        NOT NULL,
    topic       topic_tag   NOT NULL,

    CONSTRAINT pk_user_topics      PRIMARY KEY (user_id, topic),
    CONSTRAINT fk_user_topics_user FOREIGN KEY (user_id)
        REFERENCES identity.users (id) ON DELETE CASCADE
);

-- Child: PrivacySetting — exactly 5 boolean toggles (US-043)
-- Stored as one row; created atomically with User.
CREATE TABLE identity.privacy_settings (
    user_id                 UUID    NOT NULL,
    public_profile          BOOLEAN NOT NULL DEFAULT TRUE,
    show_online_status      BOOLEAN NOT NULL DEFAULT TRUE,
    show_learning_progress  BOOLEAN NOT NULL DEFAULT TRUE,
    allow_direct_messages   BOOLEAN NOT NULL DEFAULT TRUE,
    show_hourly_rate        BOOLEAN NOT NULL DEFAULT TRUE,
    updated_at              TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_privacy_settings      PRIMARY KEY (user_id),
    CONSTRAINT fk_privacy_settings_user FOREIGN KEY (user_id)
        REFERENCES identity.users (id) ON DELETE CASCADE
);

-- Child: UserAvailability (US-047 — people directory filter)
CREATE TABLE identity.user_availability (
    user_id     UUID                NOT NULL,
    slot        availability_slot   NOT NULL,

    CONSTRAINT pk_user_availability      PRIMARY KEY (user_id, slot),
    CONSTRAINT fk_user_availability_user FOREIGN KEY (user_id)
        REFERENCES identity.users (id) ON DELETE CASCADE
);

-- Outbox — transactional outbox for identity BC (Rule 4)
CREATE TABLE identity.outbox_events (
    id           BIGSERIAL   NOT NULL,
    event_type   TEXT        NOT NULL,
    aggregate_id UUID        NOT NULL,
    payload      JSONB       NOT NULL DEFAULT '{}',
    published    BOOLEAN     NOT NULL DEFAULT FALSE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT pk_identity_outbox PRIMARY KEY (id)
);

-- identity indexes
```

```
CREATE INDEX idx_users_role         ON identity.users (user_role);
CREATE INDEX idx_users_is_online    ON identity.users (is_online) WHERE is_online;
CREATE INDEX idx_users_name_trgm    ON identity.users USING GIN (display_name gin_trgm_ops);
CREATE INDEX idx_users_city_trgm    ON identity.users USING GIN (city gin_trgm_ops);
CREATE INDEX idx_ulang_user         ON identity.user_languages (user_id);
CREATE INDEX idx_ulang_code         ON identity.user_languages (language_code);
CREATE INDEX idx_outbox_id_unpub    ON identity.outbox_events (id) WHERE NOT published;
```

BC: sessions — Room, Participant, ChatMessage, HandRaise

```
-- BC: sessions
--
-- AGGREGATE ROOT 1: Room
-- True invariants (Rule 1):
--   · title 1-80 chars, description ≤ 280 chars (US-015)
--   · price_per_person present iff access = 'paid' (US-018)
--   · max_participants ≥ 2 (US-018)
--   · Status is one-way: scheduled → live → ended
--   · started_at/ended_at must be consistent with status
--
-- Child entities of Room (tightly coupled, Rule 2):
--   · RoomLanguage   — up to 3 per room (US-016)
--   · RoomLevel      — one or more CEFR levels (US-017)
--   · RoomTopic      — topic tags (US-019)
--
-- AGGREGATE ROOT 2: Participant (separate from Room — Rule 2)
-- Reason: is_muted and role change at high frequency during live
-- sessions (dozens of toggles per min). Grouping into Room would
-- cause constant optimistic-lock contention on the room row.
-- True invariant: (room_id, user_id) unique; role is singular.
--
-- AGGREGATE ROOT 3: ChatMessage (immutable after insert — Rule 1)
-- True invariant: body OR voice_url must exist; is_correction and
-- voice_url are mutually exclusive.
-- Child: ChatReaction — reactions owned by message.
--
-- AGGREGATE ROOT 4: HandRaiseRequest (Rule 2)
-- Own lifecycle: raised → approved | denied | cancelled.
-- Host resolves in a separate tx from Participant role change.
--
-- AGGREGATE ROOT 5: WaitlistEntry (trivial — unique pair)
-- AGGREGATE ROOT 6: RoomReminder  (trivial — unique pair)
-- ==============================================================

CREATE TABLE sessions.rooms (
    id                  UUID            NOT NULL DEFAULT gen_random_uuid(),
    host_id             UUID            NOT NULL, -- identity.users ID only (Rule 3)

    title               TEXT            NOT NULL
                                        CONSTRAINT ck_room_title_len
                                        CHECK (char_length(title) BETWEEN 1 AND 80),
    description         TEXT            CONSTRAINT ck_room_desc_len
                                        CHECK (description IS NULL OR char_length(description)
<= 280),

    status              room_status     NOT NULL DEFAULT 'scheduled',
    access              room_access     NOT NULL DEFAULT 'free',
    price_per_person    NUMERIC(8,2)    CONSTRAINT ck_room_price_pos
                                        CHECK (price_per_person IS NULL OR price_per_person >=
0),
    max_participants    SMALLINT        NOT NULL DEFAULT 20
                                        CONSTRAINT ck_room_max_part CHECK (max_participants >=
2),
```

```
    scheduled_at        TIMESTAMPTZ,
    estimated_duration  duration_preset,
    started_at          TIMESTAMPTZ,
    ended_at            TIMESTAMPTZ,

    -- Computed — immutable once ended_at is set (Rule 1)
    duration_seconds    INTEGER GENERATED ALWAYS AS (
        CASE WHEN started_at IS NOT NULL AND ended_at IS NOT NULL
             THEN EXTRACT(EPOCH FROM (ended_at - started_at))::INTEGER
        END
    ) STORED,

    version             INTEGER         NOT NULL DEFAULT 1, -- optimistic lock (Rule 1)
    created_at          TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ     NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_rooms              PRIMARY KEY (id),
    CONSTRAINT ck_rooms_paid_price   CHECK (
        (access = 'free'  AND price_per_person IS NULL) OR
        (access = 'paid'  AND price_per_person IS NOT NULL)
    ),
    CONSTRAINT ck_rooms_status_dates CHECK (
        (status = 'scheduled' AND started_at IS NULL     AND ended_at IS NULL    ) OR
        (status = 'live'      AND started_at IS NOT NULL AND ended_at IS NULL    ) OR
        (status = 'ended'     AND started_at IS NOT NULL AND ended_at IS NOT NULL)
    )
);

-- Child: RoomLanguage — up to 3 (US-016; application enforces max)
CREATE TABLE sessions.room_languages (
    room_id         UUID     NOT NULL,
    language_code   CHAR(3)  NOT NULL,
    language_name   TEXT     NOT NULL,
    flag_emoji      TEXT     NOT NULL,
    display_order   SMALLINT NOT NULL DEFAULT 0,

    CONSTRAINT pk_room_languages      PRIMARY KEY (room_id, language_code),
    CONSTRAINT fk_room_languages_room FOREIGN KEY (room_id)
        REFERENCES sessions.rooms (id) ON DELETE CASCADE
);

-- Child: RoomLevel (US-017)
CREATE TABLE sessions.room_levels (
    room_id  UUID       NOT NULL,
    level    cefr_level NOT NULL,

    CONSTRAINT pk_room_levels      PRIMARY KEY (room_id, level),
    CONSTRAINT fk_room_levels_room FOREIGN KEY (room_id)
        REFERENCES sessions.rooms (id) ON DELETE CASCADE
);

-- Child: RoomTopic (US-019)
CREATE TABLE sessions.room_topics (
    room_id  UUID      NOT NULL,
    topic    topic_tag NOT NULL,

    CONSTRAINT pk_room_topics      PRIMARY KEY (room_id, topic),
    CONSTRAINT fk_room_topics_room FOREIGN KEY (room_id)
        REFERENCES sessions.rooms (id) ON DELETE CASCADE
);

-- Aggregate root: Participant (separated from Room — Rule 2)
CREATE TABLE sessions.participants (
    id              UUID               NOT NULL DEFAULT gen_random_uuid(),
    room_id         UUID               NOT NULL, -- sessions.rooms ID only (Rule 3)
    user_id         UUID               NOT NULL, -- identity.users ID only (Rule 3)
    role            participant_role   NOT NULL DEFAULT 'listener',
```

```
    is_muted        BOOLEAN            NOT NULL DEFAULT FALSE,
    hand_raised_at  TIMESTAMPTZ,                 -- non-NULL = currently raised (US-026)
    joined_at       TIMESTAMPTZ        NOT NULL DEFAULT NOW(),
    left_at         TIMESTAMPTZ,
    version         INTEGER            NOT NULL DEFAULT 1, -- optimistic lock (Rule 1)

    CONSTRAINT pk_participants           PRIMARY KEY (id),
    CONSTRAINT uq_participant_room_user  UNIQUE (room_id, user_id)
);

-- Aggregate root: ChatMessage — immutable (Rule 1)
CREATE TABLE sessions.chat_messages (
    id               UUID        NOT NULL DEFAULT gen_random_uuid(),
    room_id          UUID        NOT NULL, -- sessions.rooms ID only (Rule 3)
    sender_id        UUID        NOT NULL, -- identity.users ID only (Rule 3)
    body             TEXT,
    is_correction    BOOLEAN     NOT NULL DEFAULT FALSE,
    voice_url        TEXT,
    voice_duration_s SMALLINT,
    sent_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_chat_messages        PRIMARY KEY (id),
    CONSTRAINT ck_chat_has_content     CHECK (body IS NOT NULL OR voice_url IS NOT NULL),
    CONSTRAINT ck_chat_correction_text CHECK (NOT (is_correction AND voice_url IS NOT NULL))
);

-- Child: ChatReaction (US-032)
CREATE TABLE sessions.chat_reactions (
    message_id UUID        NOT NULL,
    user_id    UUID        NOT NULL, -- identity.users ID only (Rule 3)
    emoji      TEXT        NOT NULL CONSTRAINT ck_emoji_len CHECK (char_length(emoji) BETWEEN
1 AND 8),
    reacted_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_chat_reactions         PRIMARY KEY (message_id, user_id, emoji),
    CONSTRAINT fk_chat_reactions_message FOREIGN KEY (message_id)
        REFERENCES sessions.chat_messages (id) ON DELETE CASCADE
);

-- Aggregate root: HandRaiseRequest — own lifecycle (Rule 2, US-026/030)
CREATE TABLE sessions.hand_raise_requests (
    id           UUID        NOT NULL DEFAULT gen_random_uuid(),
    room_id      UUID        NOT NULL, -- sessions.rooms ID only (Rule 3)
    requester_id UUID        NOT NULL, -- identity.users ID only (Rule 3)
    raised_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    resolved_at  TIMESTAMPTZ,
    resolution   TEXT        CONSTRAINT ck_hrr_resolution_values
                             CHECK (resolution IN ('approved','denied','cancelled')),

    CONSTRAINT pk_hand_raise_requests PRIMARY KEY (id),
    CONSTRAINT ck_hrr_resolved        CHECK (
        (resolved_at IS NULL AND resolution IS NULL) OR
        (resolved_at IS NOT NULL AND resolution IS NOT NULL)
    )
);

-- Aggregate root: WaitlistEntry (US-011)
CREATE TABLE sessions.waitlist_entries (
    id        UUID        NOT NULL DEFAULT gen_random_uuid(),
    room_id   UUID        NOT NULL, -- sessions.rooms ID only (Rule 3)
    user_id   UUID        NOT NULL, -- identity.users ID only (Rule 3)
    joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_waitlist_entries     PRIMARY KEY (id),
    CONSTRAINT uq_waitlist_room_user   UNIQUE (room_id, user_id)
);
```

```
-- Aggregate root: RoomReminder (US-012)
CREATE TABLE sessions.room_reminders (
    room_id    UUID        NOT NULL, -- sessions.rooms ID only (Rule 3)
    user_id    UUID        NOT NULL, -- identity.users ID only (Rule 3)
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_room_reminders PRIMARY KEY (room_id, user_id)
);

-- Outbox (Rule 4)
CREATE TABLE sessions.outbox_events (
    id           BIGSERIAL   NOT NULL,
    event_type   TEXT        NOT NULL,
    aggregate_id UUID        NOT NULL,
    payload      JSONB       NOT NULL DEFAULT '{}',
    published    BOOLEAN     NOT NULL DEFAULT FALSE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT pk_sessions_outbox PRIMARY KEY (id)
);

-- sessions indexes
CREATE INDEX idx_rooms_host        ON sessions.rooms (host_id);
CREATE INDEX idx_rooms_status      ON sessions.rooms (status);
CREATE INDEX idx_rooms_scheduled   ON sessions.rooms (scheduled_at) WHERE status =
'scheduled';
CREATE INDEX idx_rooms_live        ON sessions.rooms (started_at)   WHERE status = 'live';
CREATE INDEX idx_rooms_title_trgm  ON sessions.rooms USING GIN (title gin_trgm_ops);
CREATE INDEX idx_rlang_room        ON sessions.room_languages (room_id);
CREATE INDEX idx_rlang_code        ON sessions.room_languages (language_code);
CREATE INDEX idx_part_room         ON sessions.participants (room_id);
CREATE INDEX idx_part_user         ON sessions.participants (user_id);
CREATE INDEX idx_part_hand_raised  ON sessions.participants (room_id)
    WHERE hand_raised_at IS NOT NULL;
CREATE INDEX idx_chat_room_time    ON sessions.chat_messages (room_id, sent_at);
CREATE INDEX idx_chat_corrections  ON sessions.chat_messages (room_id)
    WHERE is_correction;
CREATE INDEX idx_hrr_room_open     ON sessions.hand_raise_requests (room_id)
    WHERE resolved_at IS NULL;
CREATE INDEX idx_waitlist_room     ON sessions.waitlist_entries (room_id);
CREATE INDEX idx_reminders_user    ON sessions.room_reminders (user_id);
CREATE INDEX idx_soutbox_unpub     ON sessions.outbox_events (id) WHERE NOT published;
```

BC: messaging — Conversation, DirectMessage

```
-- BC: messaging
--
-- AGGREGATE ROOT 1: Conversation
-- True invariants (Rule 1):
--    · participant_a ≠ participant_b (no self-conversations)
--    · The pair (a, b) is unique — no duplicate threads
--    · requested_by must be one of the two participants
--    · Messages blocked until request_status = 'accepted'
--
-- Child: ConversationMember — per-user read state (unread_count,
--   last_read_at). Updated in the SAME transaction as marking
--   a message read (true atomic invariant — Rule 1).
--
-- AGGREGATE ROOT 2: DirectMessage (Rule 2 — separate aggregate)
-- Reason: messages arrive at high frequency and are immutable.
-- Lumping into Conversation would lock the whole conversation
-- row on every send. Linked by conversation_id only (Rule 3).
--
-- Child: MessageReaction — owned by message.
-- ============================================================
```

```
CREATE TABLE messaging.conversations (
    id              UUID           NOT NULL DEFAULT gen_random_uuid(),
    participant_a   UUID           NOT NULL, -- identity.users ID only (Rule 3)
    participant_b   UUID           NOT NULL, -- identity.users ID only (Rule 3)
    request_status  request_status NOT NULL DEFAULT 'pending',
    requested_by    UUID           NOT NULL, -- identity.users ID only (Rule 3)
    accepted_at     TIMESTAMPTZ,
    -- Denormalised for list display (updated by domain event handler)
    last_message_at   TIMESTAMPTZ,
    last_message_body TEXT,
    version         INTEGER        NOT NULL DEFAULT 1, -- optimistic lock (Rule 1)
    created_at      TIMESTAMPTZ    NOT NULL DEFAULT NOW(),
    updated_at      TIMESTAMPTZ    NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_conversations            PRIMARY KEY (id),
    CONSTRAINT uq_conversation_pair        UNIQUE (
        LEAST   (participant_a::TEXT, participant_b::TEXT),
        GREATEST(participant_a::TEXT, participant_b::TEXT)
    ),
    CONSTRAINT ck_conv_different_users     CHECK (participant_a <> participant_b),
    CONSTRAINT ck_conv_requester_is_member CHECK (
        requested_by = participant_a OR requested_by = participant_b
    )
);

-- Child: ConversationMember (per-user read state)
CREATE TABLE messaging.conversation_members (
    conversation_id UUID        NOT NULL,
    user_id         UUID        NOT NULL, -- identity.users ID only (Rule 3)
    unread_count    INTEGER     NOT NULL DEFAULT 0
                                CONSTRAINT ck_unread_pos CHECK (unread_count >= 0),
    last_read_at    TIMESTAMPTZ,
    is_archived     BOOLEAN     NOT NULL DEFAULT FALSE,

    CONSTRAINT pk_conv_members       PRIMARY KEY (conversation_id, user_id),
    CONSTRAINT fk_conv_members_conv  FOREIGN KEY (conversation_id)
        REFERENCES messaging.conversations (id) ON DELETE CASCADE
);

-- Aggregate root: DirectMessage (Rule 2 — separated from Conversation)
CREATE TABLE messaging.direct_messages (
    id               UUID        NOT NULL DEFAULT gen_random_uuid(),
    conversation_id  UUID        NOT NULL, -- messaging.conversations ID only (Rule 3)
    sender_id        UUID        NOT NULL, -- identity.users ID only (Rule 3)
    body             TEXT,
    is_correction    BOOLEAN     NOT NULL DEFAULT FALSE,
    voice_url        TEXT,
    voice_duration_s SMALLINT,
    attachment_url   TEXT,
    attachment_name  TEXT,
    status           msg_status  NOT NULL DEFAULT 'sent',
    read_at          TIMESTAMPTZ,
    sent_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_direct_messages    PRIMARY KEY (id),
    CONSTRAINT ck_dm_has_content     CHECK (
        body IS NOT NULL OR voice_url IS NOT NULL OR attachment_url IS NOT NULL
    ),
    CONSTRAINT ck_dm_read_consistency CHECK (
        (status = 'read') = (read_at IS NOT NULL)
    )
);

-- Child: MessageReaction (US-060)
CREATE TABLE messaging.message_reactions (
    message_id UUID        NOT NULL,
```

```
    user_id    UUID        NOT NULL, -- identity.users ID only (Rule 3)
    emoji      TEXT        NOT NULL CONSTRAINT ck_dm_emoji_len CHECK (char_length(emoji)
BETWEEN 1 AND 8),
    reacted_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_message_reactions        PRIMARY KEY (message_id, user_id, emoji),
    CONSTRAINT fk_message_reactions_msg    FOREIGN KEY (message_id)
        REFERENCES messaging.direct_messages (id) ON DELETE CASCADE
);

-- Outbox (Rule 4)
CREATE TABLE messaging.outbox_events (
    id           BIGSERIAL   NOT NULL,
    event_type   TEXT        NOT NULL,
    aggregate_id UUID        NOT NULL,
    payload      JSONB       NOT NULL DEFAULT '{}',
    published    BOOLEAN     NOT NULL DEFAULT FALSE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT pk_messaging_outbox PRIMARY KEY (id)
);

-- messaging indexes
CREATE INDEX idx_conv_part_a       ON messaging.conversations (participant_a);
CREATE INDEX idx_conv_part_b       ON messaging.conversations (participant_b);
CREATE INDEX idx_conv_last_msg     ON messaging.conversations (last_message_at DESC);
CREATE INDEX idx_conv_pending      ON messaging.conversations (request_status)
    WHERE request_status = 'pending';
CREATE INDEX idx_cmember_user      ON messaging.conversation_members (user_id);
CREATE INDEX idx_cmember_unread    ON messaging.conversation_members (user_id, unread_count)
    WHERE unread_count > 0;
CREATE INDEX idx_dm_conv_time      ON messaging.direct_messages (conversation_id, sent_at);
CREATE INDEX idx_dm_sender         ON messaging.direct_messages (sender_id);
CREATE INDEX idx_dm_unread         ON messaging.direct_messages (conversation_id)
    WHERE status <> 'read';
CREATE INDEX idx_dm_corrections    ON messaging.direct_messages (conversation_id)
    WHERE is_correction;
CREATE INDEX idx_moutbox_unpub     ON messaging.outbox_events (id) WHERE NOT published;
```

BC: social — Follow, Notification, ActivityEntry

```
-- BC: social
--
-- AGGREGATE ROOT 1: Follow (Rule 1 invariant: no self-follow)
-- Creating/deleting a Follow publishes a domain event consumed
-- by reporting BC to update follower_count (eventual, Rule 4).
--
-- AGGREGATE ROOT 2: Notification (US-005)
-- Each notification has its own lifecycle (unread → read).
-- Never co-modified with other notifications.
--
-- AGGREGATE ROOT 3: ActivityEntry (US-040/041)
-- Append-only event log. Immutable after creation.
-- ==============================================================

CREATE TABLE social.follows (
    follower_id UUID        NOT NULL, -- identity.users ID only (Rule 3)
    followee_id UUID        NOT NULL, -- identity.users ID only (Rule 3)
    created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_follows            PRIMARY KEY (follower_id, followee_id),
    CONSTRAINT ck_follows_no_self    CHECK (follower_id <> followee_id)
);

CREATE TABLE social.notifications (
```

```
    id           UUID        NOT NULL DEFAULT gen_random_uuid(),
    recipient_id UUID        NOT NULL, -- identity.users ID only (Rule 3)
    actor_id     UUID,                  -- identity.users ID; NULL = system
    type         TEXT        NOT NULL  CONSTRAINT ck_notif_type CHECK (type IN (
                                 'followed_you','room_invite','mention',

'booking_request','room_starting','request_accepted','review_received'
                             )),
    entity_id    UUID,        -- loosely typed cross-BC reference (Rule 3)
    entity_type  TEXT,
    body         TEXT        NOT NULL,
    is_read      BOOLEAN     NOT NULL DEFAULT FALSE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_notifications PRIMARY KEY (id)
);

CREATE TABLE social.activity_entries (
    id          UUID        NOT NULL DEFAULT gen_random_uuid(),
    user_id     UUID        NOT NULL, -- identity.users ID only (Rule 3)
    type        TEXT        NOT NULL  CONSTRAINT ck_activity_type CHECK (type IN (
                                 'hosted_room','joined_room',
                                 'received_review','streak_milestone'
                             )),
    title       TEXT        NOT NULL,
    metadata    JSONB       NOT NULL DEFAULT '{}',
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_activity_entries PRIMARY KEY (id)
);

-- Outbox (Rule 4)
CREATE TABLE social.outbox_events (
    id           BIGSERIAL   NOT NULL,
    event_type   TEXT        NOT NULL,
    aggregate_id UUID        NOT NULL,
    payload      JSONB       NOT NULL DEFAULT '{}',
    published    BOOLEAN     NOT NULL DEFAULT FALSE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT pk_social_outbox PRIMARY KEY (id)
);

-- social indexes
CREATE INDEX idx_follows_follower   ON social.follows (follower_id);
CREATE INDEX idx_follows_followee   ON social.follows (followee_id);
CREATE INDEX idx_notifs_recipient   ON social.notifications (recipient_id, created_at DESC);
CREATE INDEX idx_notifs_unread      ON social.notifications (recipient_id) WHERE NOT is_read;
CREATE INDEX idx_activity_user_time ON social.activity_entries (user_id, occurred_at DESC);
CREATE INDEX idx_soc_outbox_unpub   ON social.outbox_events (id) WHERE NOT published;
```

BC: marketplace — TeacherProfile, Review, Booking

```
-- BC: marketplace
--
-- AGGREGATE ROOT 1: TeacherProfile (US-042)
-- True invariant: hourly_rate ≥ 0; exists only for teacher users.
-- avg_rating / review_count = eventual projection (Rule 4).
--
-- AGGREGATE ROOT 2: Review (US-045)
-- True invariants: (author, subject) unique; author ≠ subject;
-- rating 1-5.
--
-- AGGREGATE ROOT 3: Booking (US-042)
-- True invariant: teacher ≠ student; status transitions one-way.
```

```
-- ============================================================

CREATE TABLE marketplace.teacher_profiles (
    user_id      UUID         NOT NULL, -- identity.users ID only (Rule 3)
    hourly_rate  NUMERIC(8,2) NOT NULL  CONSTRAINT ck_tp_rate_pos CHECK (hourly_rate >= 0),
    is_verified  BOOLEAN      NOT NULL DEFAULT FALSE,
    headline     TEXT         CONSTRAINT ck_tp_headline_len CHECK (headline IS NULL OR
char_length(headline) <= 120),
    specialities TEXT[],
    -- Eventual projections updated by ReviewSubmitted event (Rule 4)
    avg_rating   NUMERIC(3,2) CONSTRAINT ck_tp_rating_range CHECK (avg_rating IS NULL OR
avg_rating BETWEEN 0 AND 5),
    review_count INTEGER      NOT NULL DEFAULT 0 CONSTRAINT ck_tp_review_count CHECK
(review_count >= 0),
    version      INTEGER      NOT NULL DEFAULT 1, -- optimistic lock (Rule 1)
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ  NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_teacher_profiles PRIMARY KEY (user_id)
);

CREATE TABLE marketplace.reviews (
    id         UUID        NOT NULL DEFAULT gen_random_uuid(),
    subject_id UUID        NOT NULL, -- identity.users ID only (Rule 3)
    author_id  UUID        NOT NULL, -- identity.users ID only (Rule 3)
    rating     SMALLINT    NOT NULL  CONSTRAINT ck_review_rating CHECK (rating BETWEEN 1 AND
5),
    body       TEXT        CONSTRAINT ck_review_body_len CHECK (body IS NULL OR
char_length(body) <= 500),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_reviews          PRIMARY KEY (id),
    CONSTRAINT uq_review_per_pair  UNIQUE (subject_id, author_id),
    CONSTRAINT ck_review_no_self   CHECK (subject_id <> author_id)
);

CREATE TABLE marketplace.bookings (
    id           UUID            NOT NULL DEFAULT gen_random_uuid(),
    teacher_id   UUID            NOT NULL, -- identity.users ID only (Rule 3)
    student_id   UUID            NOT NULL, -- identity.users ID only (Rule 3)
    scheduled_at TIMESTAMPTZ     NOT NULL,
    duration     duration_preset NOT NULL DEFAULT '1hour',
    amount       NUMERIC(8,2)    NOT NULL  CONSTRAINT ck_booking_amount CHECK (amount >= 0),
    currency     CHAR(3)         NOT NULL DEFAULT 'USD',
    status       TEXT            NOT NULL DEFAULT 'pending'
                                 CONSTRAINT ck_booking_status CHECK (
                                     status IN ('pending','confirmed','cancelled','completed')
                                 ),
    room_id      UUID,   -- sessions.rooms ID only (Rule 3) — set when room created
    version      INTEGER NOT NULL DEFAULT 1,  -- optimistic lock (Rule 1)
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_bookings        PRIMARY KEY (id),
    CONSTRAINT ck_booking_no_self CHECK (teacher_id <> student_id)
);

-- Outbox (Rule 4)
CREATE TABLE marketplace.outbox_events (
    id           BIGSERIAL   NOT NULL,
    event_type   TEXT        NOT NULL,
    aggregate_id UUID        NOT NULL,
    payload      JSONB       NOT NULL DEFAULT '{}',
    published    BOOLEAN     NOT NULL DEFAULT FALSE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    CONSTRAINT pk_marketplace_outbox PRIMARY KEY (id)
);
```

```
-- marketplace indexes
CREATE INDEX idx_tp_rating      ON marketplace.teacher_profiles (avg_rating DESC);
CREATE INDEX idx_tp_verified    ON marketplace.teacher_profiles (is_verified) WHERE
is_verified;
CREATE INDEX idx_reviews_subj   ON marketplace.reviews (subject_id, created_at DESC);
CREATE INDEX idx_reviews_auth   ON marketplace.reviews (author_id);
CREATE INDEX idx_book_teacher   ON marketplace.bookings (teacher_id, scheduled_at);
CREATE INDEX idx_book_student   ON marketplace.bookings (student_id, scheduled_at);
CREATE INDEX idx_book_status    ON marketplace.bookings (status)
    WHERE status IN ('pending','confirmed');
CREATE INDEX idx_mkt_outbox_unpub ON marketplace.outbox_events (id) WHERE NOT published;
```

BC: reporting — Read-model projections (eventual, Rule 4)

```
-- BC: reporting
--
-- Read-model projections (eventual — Rule 4).
-- NO domain commands are issued here.
-- Values computed from domain events published by other BCs.
-- ==============================================================

-- UserStats projection — profile stats bar (US-039) and
-- People directory sort (US-051/052)
CREATE TABLE reporting.user_stats (
    user_id         UUID          NOT NULL, -- identity.users ID only (Rule 3)
    -- Stats bar (US-039)
    sessions_count  INTEGER       NOT NULL DEFAULT 0 CONSTRAINT ck_us_sessions CHECK
(sessions_count >= 0),
    hours_spoken    NUMERIC(10,2) NOT NULL DEFAULT 0 CONSTRAINT ck_us_hours    CHECK
(hours_spoken >= 0),
    day_streak      INTEGER       NOT NULL DEFAULT 0 CONSTRAINT ck_us_streak   CHECK
(day_streak >= 0),
    longest_streak  INTEGER       NOT NULL DEFAULT 0,
    avg_rating      NUMERIC(3,2)  CONSTRAINT ck_us_rating CHECK (avg_rating IS NULL OR
avg_rating BETWEEN 0 AND 5),
    -- Social stats (US-040)
    follower_count  INTEGER       NOT NULL DEFAULT 0 CONSTRAINT ck_us_followers CHECK
(follower_count >= 0),
    following_count INTEGER       NOT NULL DEFAULT 0 CONSTRAINT ck_us_following CHECK
(following_count >= 0),
    rooms_hosted    INTEGER       NOT NULL DEFAULT 0 CONSTRAINT ck_us_hosted    CHECK
(rooms_hosted >= 0),
    rooms_joined    INTEGER       NOT NULL DEFAULT 0 CONSTRAINT ck_us_joined    CHECK
(rooms_joined >= 0),
    last_session_at TIMESTAMPTZ,
    updated_at      TIMESTAMPTZ   NOT NULL DEFAULT NOW(),

    CONSTRAINT pk_user_stats PRIMARY KEY (user_id)
);

-- SessionLog — source of truth for streak and hours computation
CREATE TABLE reporting.session_logs (
    id               BIGSERIAL        NOT NULL,
    user_id          UUID             NOT NULL, -- identity.users ID only (Rule 3)
    room_id          UUID             NOT NULL, -- sessions.rooms  ID only (Rule 3)
    role             participant_role NOT NULL,
    joined_at        TIMESTAMPTZ      NOT NULL,
    left_at          TIMESTAMPTZ,
    duration_seconds INTEGER GENERATED ALWAYS AS (
        CASE WHEN left_at IS NOT NULL
             THEN EXTRACT(EPOCH FROM (left_at - joined_at))::INTEGER
        END
    ) STORED,
```

```
    CONSTRAINT pk_session_logs PRIMARY KEY (id)
);

-- reporting indexes
CREATE INDEX idx_stats_streak    ON reporting.user_stats (day_streak DESC);
CREATE INDEX idx_stats_sessions  ON reporting.user_stats (sessions_count DESC);
CREATE INDEX idx_stats_rating    ON reporting.user_stats (avg_rating DESC);
CREATE INDEX idx_slog_user       ON reporting.session_logs (user_id, joined_at DESC);
CREATE INDEX idx_slog_room       ON reporting.session_logs (room_id);
```

Cross-BC Read Model View + Triggers

```
-- CROSS-BC READ MODELS (query layer — Rule 3)
-- Joins across schemas happen ONLY here, never in command handlers.
-- ==============================================================

-- People directory view (US-046-053)
CREATE VIEW identity.people_directory AS
SELECT
    u.id, u.handle, u.display_name, u.city, u.bio,
    u.avatar_url, u.user_role, u.is_online, u.last_seen_at, u.created_at,
    (
        SELECT JSON_AGG(JSON_BUILD_OBJECT(
            'code', ul.language_code, 'name', ul.language_name,
            'flag', ul.flag_emoji, 'level', ul.level,
            'is_native', ul.is_native, 'order', ul.display_order
        ) ORDER BY ul.display_order)
        FROM identity.user_languages ul WHERE ul.user_id = u.id
    ) AS languages,
    (SELECT ARRAY_AGG(ut.topic)   FROM identity.user_topics      ut WHERE ut.user_id = u.id)
AS topics,
    (SELECT ARRAY_AGG(ua.slot)    FROM identity.user_availability ua WHERE ua.user_id = u.id)
AS availability,
    s.sessions_count, s.hours_spoken, s.day_streak, s.avg_rating,
    s.follower_count, s.following_count, s.rooms_hosted,
    tp.hourly_rate, tp.is_verified, tp.avg_rating AS teacher_rating, tp.review_count
FROM identity.users                    u
LEFT JOIN reporting.user_stats         s  ON s.user_id  = u.id
LEFT JOIN marketplace.teacher_profiles tp ON tp.user_id = u.id;

-- ==============================================================
-- TRIGGERS
-- ==============================================================

-- updated_at maintenance
CREATE OR REPLACE FUNCTION public.touch_updated_at()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN NEW.updated_at = NOW(); RETURN NEW; END;
$$;

CREATE TRIGGER trg_users_updated_at
    BEFORE UPDATE ON identity.users
    FOR EACH ROW EXECUTE FUNCTION public.touch_updated_at();

CREATE TRIGGER trg_rooms_updated_at
    BEFORE UPDATE ON sessions.rooms
    FOR EACH ROW EXECUTE FUNCTION public.touch_updated_at();

CREATE TRIGGER trg_conversations_updated_at
    BEFORE UPDATE ON messaging.conversations
    FOR EACH ROW EXECUTE FUNCTION public.touch_updated_at();
```

```
CREATE TRIGGER trg_teacher_profiles_updated_at
    BEFORE UPDATE ON marketplace.teacher_profiles
    FOR EACH ROW EXECUTE FUNCTION public.touch_updated_at();

CREATE TRIGGER trg_bookings_updated_at
    BEFORE UPDATE ON marketplace.bookings
    FOR EACH ROW EXECUTE FUNCTION public.touch_updated_at();

-- Optimistic concurrency — enforce version increment on writes (Rule 1)
CREATE OR REPLACE FUNCTION public.enforce_version_bump()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.version <= OLD.version THEN
        RAISE EXCEPTION
            'Stale aggregate write: expected version > %, received %',
            OLD.version, NEW.version
        USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_users_version
    BEFORE UPDATE ON identity.users
    FOR EACH ROW EXECUTE FUNCTION public.enforce_version_bump();

CREATE TRIGGER trg_rooms_version
    BEFORE UPDATE ON sessions.rooms
    FOR EACH ROW EXECUTE FUNCTION public.enforce_version_bump();

CREATE TRIGGER trg_participants_version
    BEFORE UPDATE ON sessions.participants
    FOR EACH ROW EXECUTE FUNCTION public.enforce_version_bump();

CREATE TRIGGER trg_conversations_version
    BEFORE UPDATE ON messaging.conversations
    FOR EACH ROW EXECUTE FUNCTION public.enforce_version_bump();

CREATE TRIGGER trg_teacher_profiles_version
    BEFORE UPDATE ON marketplace.teacher_profiles
    FOR EACH ROW EXECUTE FUNCTION public.enforce_version_bump();

CREATE TRIGGER trg_bookings_version
    BEFORE UPDATE ON marketplace.bookings
    FOR EACH ROW EXECUTE FUNCTION public.enforce_version_bump();
```

Part 3 — User Story → Table Traceability

Every user story traces to at least one table, column, or constraint. The matrix below maps key stories to their primary schema artefacts.

US	Feature	Primary Tables / Columns	DDD artefact
US-002	Unread badge on nav	`messaging.conversation_members.unread_count`	Projection (Rule 4)
US-005	Notification bell + dot	`social.notifications (is_read, type, body)`	Aggregate root: Notification
US-011	Join waitlist	`sessions.waitlist_entries (room_id, user_id)`	Aggregate root: WaitlistEntry
US-012	Room reminder	`sessions.room_reminders (room_id, user_id)`	Aggregate root: RoomReminder
US-015	Title 80 / desc 280 chars	`rooms.title CHECK, rooms.description CHECK`	Rule 1 invariant constraint
US-016	Up to 3 languages	`sessions.room_languages (room_id, language_code)`	Child entity of Room
US-017	CEFR level selection	`sessions.room_levels (room_id, level)`	Child entity of Room
US-018	Paid access + price	`rooms.access, rooms.price_per_person, ck_rooms_paid_price`	Rule 1 invariant: paid ↔ price
US-019	Topics + scheduling	`sessions.room_topics, rooms.scheduled_at, rooms.estimated_duration`	Child entity + Value Object
US-026	Hand raise	`sessions.participants.hand_raised_at + hand_raise_requests`	Two separate aggregates (Rule 2)
US-030	Host approves hand raise	`sessions.hand_raise_requests.resolution, sessions.participants.role`	Separate lifecycle (Rule 2)
US-031	Room chat	`sessions.chat_messages (body, is_correction, voice_url, sent_at)`	Aggregate root: ChatMessage
US-032	Chat reactions	`sessions.chat_reactions (message_id, user_id, emoji)`	Child entity of ChatMessage
US-036	Bio edit (220 chars)	`identity.users.bio CHECK char_length(bio) <= 220`	Rule 1 invariant
US-037	Language progress bars	`identity.user_languages (level, progress_pct, months_studied, is_native)`	Child entity of User
US-038	Topic chips	`identity.user_topics (user_id, topic)`	Child entity of User
US-039	Stats bar	`reporting.user_stats (sessions_count, hours_spoken, day_streak, avg_rating)`	Read model (Rule 4)
US-040	Social stats + activity	`reporting.user_stats + social.activity_entries`	Projection + Event log
US-042	Teacher card	`marketplace.teacher_profiles (hourly_rate, is_verified, avg_rating)`	Aggregate root: TeacherProfile
US-043	5 privacy toggles	`identity.privacy_settings (5 BOOLEAN columns)`	Child entity of User
US-044	Follow / Unfollow	`social.follows (follower_id, followee_id, ck_follows_no_self)`	Aggregate root: Follow
US-045	Reviews	`marketplace.reviews (rating CHECK, uq_review_per_pair, ck_review_no_self)`	Aggregate root: Review

US	Feature	Primary Tables / Columns	DDD artefact
US-047	People filter	identity.user_availability (slot enum) + identity.people_directory view	Child entity + ACL view
US-056	Auto-mark read	messaging.conversation_members.unread_count, last_read_at	Child entity of Conversation
US-058	Read receipts	messaging.direct_messages.status, read_at (ck_dm_read_consistency)	Rule 1 invariant
US-059	Corrections + voice	direct_messages.is_correction, voice_url, voice_duration_s	Polymorphic content columns
US-060	DM reactions	messaging.message_reactions (message_id, user_id, emoji)	Child entity of DirectMessage
US-062	File + voice attach	messaging.direct_messages.attachment_url, attachment_name	Polymorphic content columns
US-064	Accept/decline request	messaging.conversations.request_status, accepted_at	Rule 1 status invariant